



Welcome to CSC 365

Internet Programming

Dr. Yifan Zhang

How do you program in JavaScript?

From Wikipedia:

...

... supporting object-oriented, imperative, and functional programming

...

- Originally programming conventions (i.e. patterns) rather than language features
 - ECMAScript adding language features (e.g. class, => , etc.)

Object-oriented programming: methods

- With first class functions a property of an object can be a function

```
let obj = {count: 0};  
obj.increment = function (amount) {  
    this.count += amount;  
    return this.count;  
}
```

- Method invocation: calls function and binds this to be object

```
obj.increment(1); // returns 1  
obj.increment(3); // returns 4
```

This

- With first class functions a property of an object can be a function

```
let o = { oldProp: 'this is an old property' };  
o.aMethod = function() {  
  this.newProp = "this is a new property";  
  return Object.keys(this);  
}  
o.aMethod();
```

- In non-method functions:
 - this will be the global object
 - Or if "use strict"; this will be undefined

functions are objects - can have properties

```
function plus1(value) {  
    if (plus1.invocations == undefined) {  
        plus1.invocations = 0;  
    }  
    plus1.invocations++;  
    return value + 1;  
}
```

- plus1.invocations will be the number times function is called
- Acts like static/class properties in object-oriented languages

function are objects: Have methods

```
function func(arg) { console.log(this,arg); }
```

- `toString()` method - return function as source string
 - `func.toString()` returns `'function func(arg) { console.log(this,arg); }'`
- `call()` method - call function specifying this and arguments:
 - `func.call({t: 1}, 2)` prints `'{ t: 1 } 2'`
 - `apply()` like `call()` except arguments are passed as an array -
`func.apply({t: 2},[2])`
 - this is like an extra hidden argument to a function call and is used that way sometimes
- `bind()` method - creates a new function with this and arguments bound
 - `let newFunc = func.bind({z: 2}, 3);`
 - `newFunc()` prints `'{ z: 2 } 3'`

Object-oriented programming: classes

Functions are classes in JavaScript: Name the function after the class

```
function Rectangle(width, height) {  
    this.width = width;  
    this.height = height;  
    this.area = function() {  
        return this.width*this.height;  
    }  
}
```

```
let r = new Rectangle(26, 14); // {width: 26, height: 14}
```

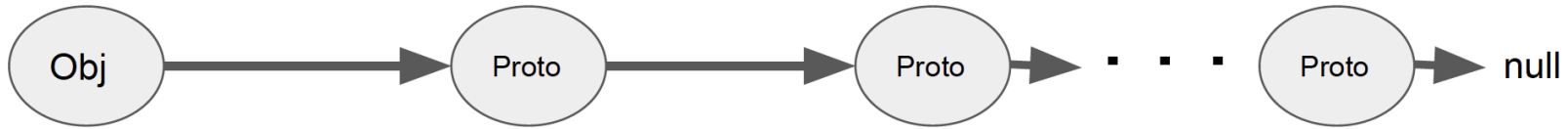
Functions used in this way are called constructors:

```
r.constructor.name == 'Rectangle'
```

```
console.log(r.area()); // 364  
console.log(r.constructor.name); // "Rectangle"
```

Object-oriented programming: inheritance

- Javascript has the notion of a prototype object for each object instance
 - Prototype objects can have prototype objects forming a prototype chain



- On an object property read access JavaScript will search the up the prototype chain until the property is found
- Effectively the properties of an object are its own property in addition to all the properties up the prototype chain. This is called prototype-based inheritance.
- Property updates are different: always create property in object if not found

Using prototypes

```
function Rectangle(width, height) {  
  this.width = width;    // own (instance) properties  
  this.height = height;  
}
```

```
Rectangle.prototype.area = function () {  
  return this.width * this.height;  
};
```

```
let r = new Rectangle(26, 14);  
let v = r.area();           // 26 * 14  
Object.keys(r);             // ['width', 'height'] (only own props)
```

Note: Dynamic - changing prototype will cause all instances to change⁹

Prototype versus object instances

```
let r = new Rectangle(26, 14);
```

Understand the difference between:

```
r.newMethod = function () {  
  console.log('New Method called');  
};
```

newMethod becomes an own property of r, Only r can call it; another instance won't have it:

```
let s = new Rectangle(1, 1);  
s.newMethod; // undefined
```

Prototype versus object instances

```
let r = new Rectangle(26, 14);
```

And:

```
Rectangle.prototype.newMethod = function () {  
  console.log('New Method called');  
};
```

newMethod is on Rectangle.prototype, so all instances—existing and future—can call it:

```
r.newMethod(); // works  
s.newMethod(); // now works too
```

Inheritance

`Rectangle.prototype = new Shape(...);`

- If desired property not in `Rectangle.prototype` then JavaScript will look in `Shape.prototype` and so on.
 - Can view prototype objects as forming a chain. Lookups go up the prototype chain.
- Prototype-based inheritance
 - Single inheritance support
 - Can be dynamically created and modified

ECMAScript version 6 extensions

```
class Rectangle extends Shape {  
  constructor(height, width) {  
    super(height, width);  
    this.height = height;  
    this.width = width;  
  }  
  
  area() {  
    return this.width * this.height;  
  }  
  
  static countRects() {  
    ...  
  }  
}  
  
let r = new Rectangle(10, 20);
```

// inheritance: Rectangle → Shape
// runs when you do `new Rectangle(...)`
// call parent (Shape) constructor
// instance (own) properties

// prototype method (shared by instances)

// static method (on the class, not instances)

React.js example class

```
class HelloWorld extends React.Component {  
  constructor(props) {  
    super(props);          // must call super(props) before using `this`  
    // you could initialize state here: this.state = { ... }  
  }  
  
  render() {               // required method in class components  
    return (  
      <div>Hello World</div> // JSX → transpiles to React.createElement(...)  
    );  
  }  
}
```

Functional Programming

- Imperative:

```
let newArr = [];  
for (let i = 0; i < anArr.length; i++) {  
    newArr[i] = anArr[i] * i;    // explicitly index and assign (mutation)  
}
```

- Functional:

```
const newArr = anArr.map((val, ind) => val * ind);
```

- Can write entire program as functions with no side-effects

```
const result =  
    anArr  
        .filter(filterFunc)    // keep certain elements  
        .map(mapFunc)          // transform each element  
        .reduce(reduceFunc);   // aggregate into one value
```

We can mostly but not totally avoid functional style

- Asynchronous events done with callback functions

Browser Example:

```
function callbackFunc() { console.log("timeout"); }  
setTimeout(callbackFunc, 3 * 1000); // schedule a function to run later
```

Server:

```
function callbackFunc(err, data) { console.log(String(data)); }  
fs.readFile('/etc/passwd', callbackFunc); // async I/O with error-first callback
```

- Node.js programming: Write function for HTTP request processing
- React's JSX prefers functional style: map(), filter(), ?:

Closures

An advanced programming language concept you need to know about

What happens if a call
myFunc()? Again?

- myFunc closure contains
argVar, localVar and
globalVar

```
let globalVar = 1;

function localFunc(argVar) {
  let localVar = 0;

  function embedFunc() {
    return ++localVar + argVar + globalVar;
  }

  return embedFunc;
}

let myFunc = localFunc(10);
```

Using Scopes and Closures

- Using scopes and closures to create modules in early JavaScript

```
var i = 1;  
function f() {  
  i++;  
  return i;  
}
```

```
return (function () {  
  var i = 1;          // private  
  function f() { i++; return i; }  
  return f;           // expose only what you want  
})();
```

Using Scopes and Closures

Modern equivalents

```
const makeCounter = () => {  
  let i = 1;  
  return () => ++i;  
};  
  
const counter = makeCounter();  
counter(); // 2  
counter(); // 3
```

Using closures for private object properties

```
let myObj = (function () {  
  let privateProp1 = 1;           // private (not on the returned object)  
  let privateProp2 = "test";      // private  
  
  let setPrivate1 = function (val1) { privateProp1 = val1; }; // public setter  
  let compute = function () { return privateProp1 + privateProp2; }; // public method  
  
  return { compute, setPrivate1 }; // public API (no direct access to the privates)  
})();
```

typeof myObj; // 'object'

Object.keys(myObj); // ['compute', 'setPrivate1']

What does myObj.compute() return?

JavaScript Object Notation (JSON)

```
const obj = { ps: 'str', pn: 1, pa: [1, 'two', 3, 4], po: { sop: 1 } };  
const s = JSON.stringify(obj);  
// s === '{"ps":"str","pn":1,"pa":[1,"two",3,4],"po":{"sop":1}}'  
typeof s; // 'string'
```

- JSON is the standard format for sending data to and from a browser
- JSON string to JavaScript object: `JSON.parse()`

In Class Activity 4

Functions

Next Class



Week 3:

Topic:

- Javascript